M2104 Conception Objets – Notes de cours – Élaborer un programme
Session 5

1 Cours : sous-typage comportemental, polymorphisme, interfaces et héritage

1.1 Le sous-typage comportemental

En programmation, les individus (variables, constantes, paramètres, attributs...) sont d'un certain *type* (entier, chaîne de caractères, etc.). Ce type définit les opérations disponibles sur cet individu (les opérations arithmétiques sur les nombres avec une différence pour la division euclidienne, chez les entiers, et à virgule flottante, chez les nombres à virgule flottante). La notion de type est vaste, et elle comprend notamment la notion de *sous-type* : intuitivement, toutes les voitures sont des véhicules, mais tous les véhicules ne sont pas des voitures, le type *voiture* est donc un sous-type du type *véhicule*. Le *sous-typage comportemental* définit une relation dans laquelle tous les individus du sous-type d'un même type partagent un même *comportement* (tous les véhicules se déplacent), qui est exécuté différemment suivant les sous-types (une voiture ne se déplace pas comme un vélo).

Dans le paradigme *objets* de la programmation, les classes peuvent participer à ces relations de sous-typage ; en Java, les mécanismes le permettant comprennent les **interfaces**, les **classes abstraites**, l'**héritage**. Comme un objet peut *prendre plusieurs formes*, le terme générique employé pour ces pratiques est le **polymorphisme**.

Avant de détailler *comment* mettre en place le polymorphisme, voyons rapidement *pourquoi*.

1.2 Objectif : réduire les redondances

C'est l'un des premiers constats lors de l'apprentissage de la programmation : on écrit souvent du code très similaire, voire le même code avec de petites variations. Pour éviter de perdre du temps à refaire la même chose, le programmeur utilise des *fonctions* qui lui permettent de reproduire des tâches courantes (comme chercher un élément dans un tableau) avec les éléments variables en paramètre de ces fonctions.

De même, le polymorphisme permet d'éviter de dupliquer du code similaire, voire identique entre classes. Si deux méthodes sont identiques, si deux classes ont presque les mêmes attributs, il est souhaitable de minimiser ces redondances, non seulement pour éviter de faire deux fois le même travail, mais surtout pour permettre de corriger, tester, maintenir ce qui a été fait plus facilement.

1.3 Objectif : avoir différents comportements pour une collection d'objets

Si plusieurs classes disposent d'une même opération, comme le fait d'avoir une représentation textuelle (`toString`), et qu'on souhaite itérer cette opération pour l'ensemble de ces objets, disposer d'un type unifiant tous ces éléments et disposant de cette opération permet de les rassembler tous dans une collection d'objets de ce type, et d'appeler l'opération voulue sur cette collection. L'opération voulue ainsi différente pour chaque objet suivant son type spécifique, mais appliquée pour tous comme exigé par le type général.

1.4 Objectif : s'abstraire des spécificités d'un code

Pouvoir appliquer une opération sur un objet en n'utilisant que la **déclaration** de la méthode correspondante, c'est-à-dire sa **signature** (le nom et la liste des types de ses paramètres) permet, en plus d'avoir un comportement différent si les implémentations de cette méthode diffèrent dans les classes visées, de s'abstraire de ces implémentations. L'**abstraction** est le fait de ne pas dépendre des spécificités du code d'une méthode, mais seulement de cette signature et d'une spécification des actions effectuées, qui peut prendre la forme de tests et de documentation.

En Java, l'abstraction est réalisée sous la forme de **méthodes abstraites** et/ou d'**interfaces**. Ces éléments constituent un **contrat de programmation** : en les déclarant, tout programmeur qui crée une classe qui soit un sous-type d'une interface ou d'une classe comportant des méthodes abstraites s'engage à respecter les spécification données sous forme de signature de méthode, en **fournissant le code d'une méthode ayant cette signature**. La programmation par contrat permet, en établissant les contrats préalablement au code, de s'appuyer sur des méthodes **avant** de les coder, et donc de ne pas dépendre de leur bonne implémentation.

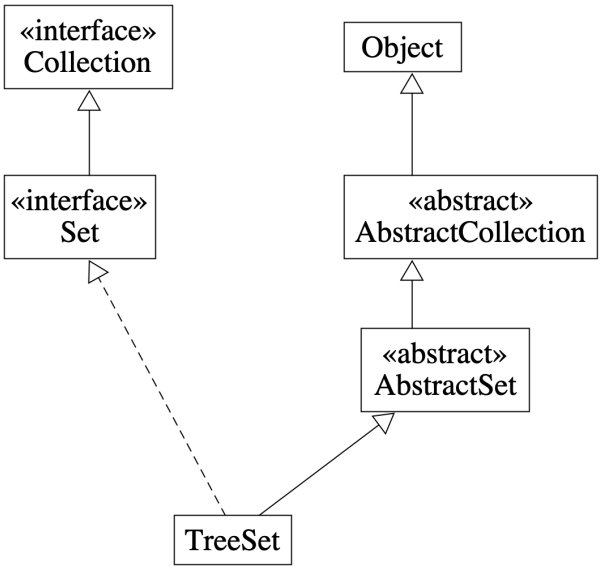
1.5 Une organisation des classes avec abstraction, polymorphisme et héritage

En Java, ces mécanismes permettent une organisation dans laquelle classes et interfaces forment une **hiérachie de types**.

Les interfaces sont des contrats de programmation qui définissent des opérations sous la forme de **signature de méthodes**, sans y associer de données ou de code concret, elles ne peuvent être directement instanciées. Des classes peuvent **se conformer** à des interfaces, en déclarant respecter leur contrat (le mot-clé Java est *implements*), ce qui fait de ces classes des sous-types des interfaces. Certaines classes peuvent aussi définir des **méthodes abstraites**, avec une simple signature ; en Java, ce sont les **classes abstraites** (mot-clé *abstract*), qui ne peuvent non plus être instanciées, mais qui peuvent également contenir des méthodes avec leur code et des attributs. Les classes peuvent enfin **hériter** d'autres classes (mot-clé Java : *extends*), ce qui en fait un sous-type. En Java, une classe ne peut hériter (directement) que d'une autre classe, mais peut se conformer à autant d'interfaces que souhaité. Les classes abstraites doivent être héritées, mais les classes non abstraites peuvent être héritées également.

Les interfaces peuvent également hériter d'autres interfaces. Les relations de sous-typage forment une hiérarchie de classes (et d'interfaces) semblable à une **taxonomie** [1] : chaque individu d'un type est membre de ce type, mais également de tous ses super-types.

Par exemple, dans un extrait de la hiérarchie des collections en Java : un `TreeSet` est un `Set`, une `Collection`, et un `Object` (entre autres, ce diagramme est partiel) :



1.6 Détail d'une mécanique objet : les interfaces

Dans le détail, les **interfaces** en Java sont, comme les classes, définies dans un fichier portant leur nom. Elles *définissent des opérations possibles* sur des objets, et donc ce qu'un objet peut faire : dans certaines conventions, les interfaces peuvent être désignées par des adjectifs comme *Affichable*, *Supprimable*, *Déplaçable*...

Le contenu d'une interface est simplement une liste de **signatures de méthodes** avec leur nom, type de valeur de retour, et liste de paramètres (et un `';'` en lieu et place du code des méthodes). Il peut y avoir également des **attributs ou méthodes d'interface** (`static`), non associés à des objets, et, depuis Java 8, des *méthodes par défaut* (ce mécanisme n'est pas détaillé ici).

Le principe est que les signatures des méthodes (dont la visibilité est publique par défaut sans que ce mot-clé doive être précisé) définit les opérations que doivent obligatoirement implémenter les classes **concrètes** qui la réalisent (les classes abstraites peuvent déléguer à leurs sous-classes cette implémentation). Tout **objet** du type de cette interface doit pouvoir effectuer les opérations déclarées dans celle-ci, et donc avoir un type plus spécifique (une classe concrète) qui donne le **code** à utiliser.

En Java, se conformer à une interface, pour une classe, se fait avec le mot-clé *implements* suivi du nom de l'interface. Pour **réaliser** cette interface, il faut également donner le code de chaque méthode déclarée dans l'interface. On utilise l'annotation `@Override`, précédent la définition de la méthode, pour indiquer que cette méthode a été déclarée dans un super-type.

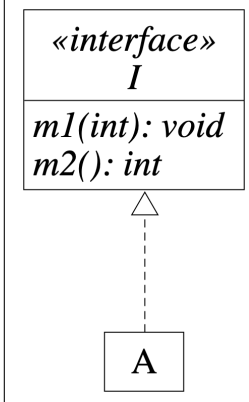
La notation utilisée en UML est un **trait en pointillés terminé par un triangle isocèle vide** placé du côté de l'interface, la reliant à la classe qui s'y conforme (ne pas confondre avec le trait en pointillés terminé par une flèche, dénotant une relation d'usage...). Les interfaces sont définies comme les classes, dans des rectangles ; leur texte est en *italique* (toute méthode abstraite est normalement en italique en UML), et leur nom est précédé du stéréotype *interface*. Toute classe concrète devant obligatoirement implémenter les méthodes déclarées dans l'interface à laquelle elle se conforme, **le diagramme de classes UML ne répète pas ces méthodes**, ce qui serait redondant (la même information ne se trouve pas deux fois dans un diagramme UML).

Les interfaces peuvent **hériter d'autres interfaces**. Dans ce cas, la plus spécifique comporte aussi comme spécification l'ensemble de toutes les méthodes déclarées plus haut dans sa hiérarchie.

1. Une taxonomie est une arborescence de termes représentant des relations *est un* – par exemple, la taxonomie du vivant distingue les animaux, végétaux, champignons... puis détaille les ordres, genres, espèces, etc.

Voici un exemple :

```
interface I {
  void m1(int a);
  int m2();
}
public class A implements I {
  private int x=14;
  @Override
  public void m1(int y) {
    x=y;
  }
  @Override
  public int m2() {
   return x++;
  }
}
```



1.7 Détail d'une mécanique Java : les interfaces fonctionnelles

En Java, pour avoir un objet du type d'une interface, on peut soit écrire la déclaration complète d'une classe concrète se conformant à cette interface puis instancier cette dernière, soit utiliser une **classe anonyme** (détails, peut-être, en cours de programmation objet). Une catégorie d'interfaces Java rend les choses plus aisées pour le programmeur, ce sont les **interfaces fonctionnelles**.

Une interface fonctionnelle est simplement une interface qui ne déclare qu'*une* unique méthode (hors méthodes d'interface ou par défaut), il suffit donc que la classe concrète la réalisant donne le code de cette méthode.

En Java, il est possible de donner le code de cette méthode sous la forme d'une **λ-expression** (aussi appelée *fonction anonyme*). Par exemple, pour un prédicat pouvant être utilisé dans la méthode `filter`, on peut utiliser les codes suivants qui sont équivalents :

```
// 1
Predicate<String> fourChars = new FourChars();
public class FourChars implements Predicate<String> {
    @Override
    public boolean test(String s) {
        return s!=null && s.length()==4;
      }
}
// 2
Predicate<String> fourChars = new Predicate<String>() {
    @Override
    public boolean test(String s) {
        return s!=null && s.length()==4;
      }
}
// 3
Predicate<String> fourChars = s -> s!=null && s.length()==4;
```

1.8 Détail d'une mécanique objet : les classes abstraites

Les **classes abstraites** en Java sont des classes auxquelles on ajoute le mot-clé **abstract**. Ces classes **ne peuvent pas être directement instanciées**, on ne peut construire d'objets de ce type qu'en utilisant une sous-classe concrète (non-abstraite), comme pour les interfaces. Elles **peuvent** comporter des méthodes abstraites, signalées par le même mot-clé (*abstract*) (une classe abstraite en Java peut ne pas déclarer de méthodes abstraites, mais toute classe qui en déclare est forcément abstraite). Elles peuvent aussi contenir les mêmes éléments que les autres classes (méthodes non abstraites avec leur code, attributs, constructeurs, attributs ou méthodes de classe).

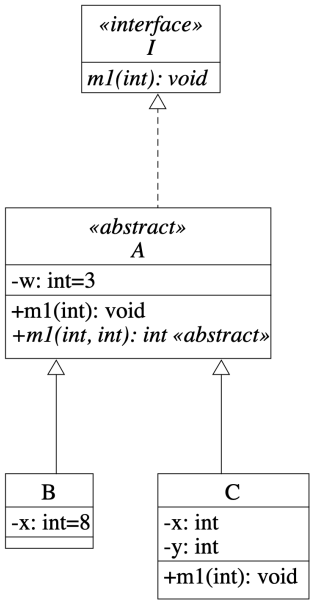
Un des avantages des classes abstraites est la *factorisation du code* entre classes fournissant le même ensemble d'opérations (donc pouvant réaliser une même interface), quand certaines ont beaucoup d'éléments communs et ne diffèrent que par une implémentation différente d'une méthode ; les données, autres méthodes et même le code du constructeur peuvent être regroupés au sein d'une classe abstraite.

Pour permettre de créer un type concret (avec possibilité créer des objets directement de ce type) qui soit un sous-type d'une classe abstraite, une classe doit en **hériter**, en Java, en ajoutant le mot-clé *extends* et le nom de la classe. Cette classe doit ensuite donner une implémentation à toutes les méthodes abstraites définies dans les classes dont elle hérite. On utilise l'annotation `@Override`, comme pour les interfaces. La classe concrète peut également ajouter d'autres éléments (attributs, méthodes, etc.).

Dans le diagramme de classes UML, les classes abstraites sont signalées par le stéréotype *abstract* noté (ainsi que le nom de la classe) en *italique*. Les méthodes abstraites sont également en italique, et suivies du stéréotype *abstract* (c'est en fait optionnel en UML, mais la réalisation de diagrammes sur papier rend l'italique peut discernable en pratique). L'héritage est noté avec un **trait continu terminé par un triangle isocèle vide** du côté de la super-classe (ne pas confondre avec le trait continu terminé par une flèche, qui dénote une association avec indication de navigabilité). Comme pour les interfaces, les méthodes dont la définition est rendue nécessaire par le fait d'hériter d'une classe les déclarant en tant que méthodes abstraites ne sont pas répétées dans le diagramme UML de cette classe, ce serait redondant.

Voici un exemple :

```
interface I {
  void m1(int a);
}
public abstract class A implements I {
  private int w=3;
  @Override
  public void m1(int y) {
    w=y;
  }
  public abstract int m1(int a, int b);
}
public class B extends A {
  private int x=8;
  @Override
  public int m1(int a, int c) {
    x+=a;
    m1(c);
    return x+c;
  }
}
public class C extents A {
  private int x, y;
  @Override
  public void m1(int z) {
    x=y=z;
  }
  @Override
  public int m1(int k, int v) {
    return k-v;
  }
}
```



Ne pas confondre **définition ou redéfinition** (*override*, soit *outrepasser*), l'acte de donner une définition, un code, une implémentation (que ce soit la première ou une nouvelle) à une méthode déclarée ou définie par un super-type, et **surcharger** (*overload*, littéralement *surcharger*), l'acte de créer une méthode du même nom qu'une déjà existante mais avec une autre liste d'arguments.

1.9 Détail d'une mécanique objet : l'héritage

L'**héritage** n'est pas réservé aux classes abstraites. Les classes peuvent être héritées (termes souvent utilisés : *étendues*, *spécialisées*...) pour y apporter d'autres fonctionnalités, s'approprier du code défini ailleurs (dans un autre paquet ou une bibliothèque externe), ou pour toute autre raison.

L'héritage entre classes permet de **redéfinir** certaines méthodes déjà définies dans les classes dont on hérite, ce qui a pour effet d'avoir la même opération (méthode) avec un comportement différent (code). On peut également *ajouter* des données (attributs) ou opérations (méthodes) supplémentaires.

Pour faciliter l'utilisation des attributs avec d'éventuelles sous-classes, la visibilité de ces attributs peut être *restreinte au paquet* (la visibilité par défaut de Java), ce qui les rend accessibles aux autres classes du paquet y compris les sous-classes, ou plus souvent *protégée* (en Java *protected*), ce qui les rend accessibles à toute classe qui en héritera (*y compris à l'extérieur du paquet*). Cette hiérarchie est celle de Java, d'autres langages peuvent avoir d'autres structures. La visibilité protégée est notée # dans les diagrammes UML, c'est l'une des plus courantes quand l'attribut est destiné à être utilisé par des sous-classes.

Le mot-clé `super` en Java permet de faire appel à la super-classe immédiate d'une classe. On peut ainsi :

— utiliser un attribut (accessible) de la super-classe immédiate (utile quand un attribut de la classe porte le même nom),

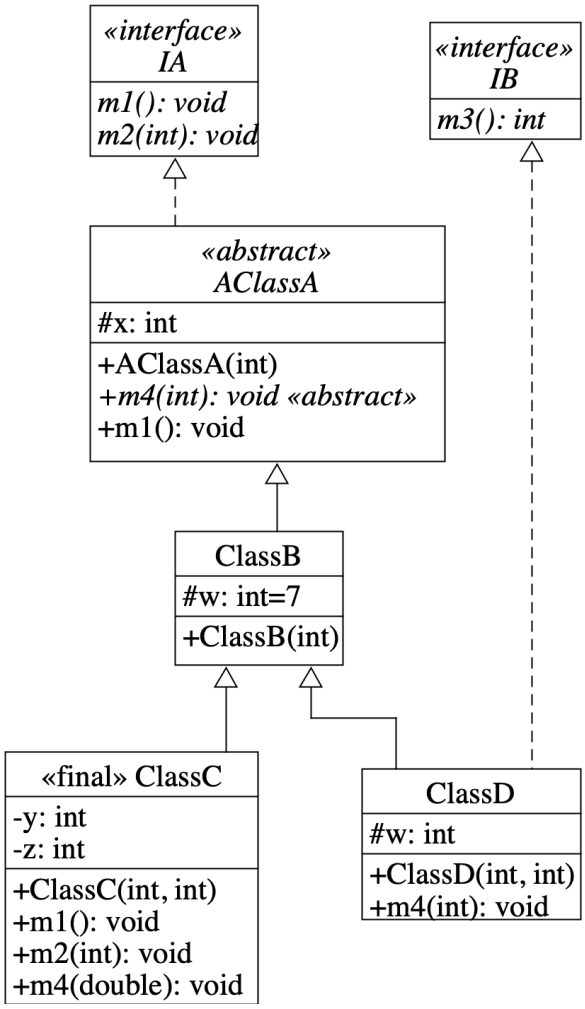
— utiliser une méthode de la super-classe immédiate,

— utiliser le constructeur de la super-classe immédiate.

Pour empêcher l'héritage, le programmeur peut déclarer sa classe comme étant terminale (en Java `final`).

En général, l'utilisation de classes abstraites et d'interfaces est à privilégier, car hériter d'une classe complexe lie directement la sous-classe à l'ensemble du code défini dans celle dont elle hérite.

Voici un exemple commenté de classes avec interfaces, classe abstraite et héritage correspondant au diagramme suivant :



`interface IA {`
`  void m1();`
`  void m2(int i);`
`}`
	`IA` déclare deux méthodes, `m1(): void` et `m2(int): void`.

`interface IB {`
`  int m3();`
`}`
	`IB` déclare une méthode, `m3(): int`.

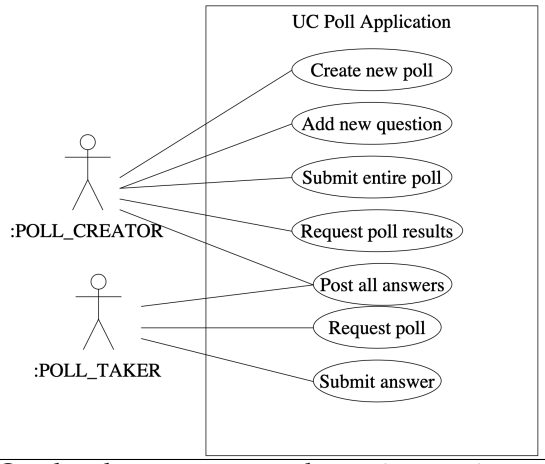
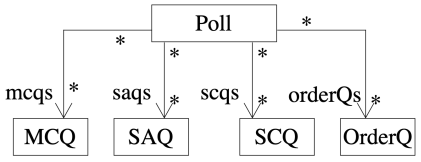
`public abstract class AClassA implements IA {`
`  protected int x;`
`  public AClassA(int x) { this.x = x; }`
`  public abstract void m4(int i);`
`  @Override`
`  public void m1() { x++; }`
`}`
	`AClassA` est abstraite. Bien qu'elle déclare se conformer au contrat défini par `IA`, elle n'est pas tenue d'implémenter l'ensemble des méthodes qui y sont déclarées. Elle donne une implémentation (définition) pour `m1(): void` et ajoute la déclaration d'une autre méthode abstraite, `m4(int): void`. Elle ajoute également un attribut protégé et un constructeur.

`public class ClassB extends AClassA {`
`  protected int w=7;`
`  public ClassB(int x) { super(x/2); }`
`  @Override`
`  public void m2(int a) { x+= a; }`
`  @Override`
`  public void m4(int b) {x -=b+w; }`
	`ClassB` n'est pas abstraite, elle doit donc fournir des implémentations (définitions) pour `m2(int): void` (déclarée dans l'interface) et `m4(int): void` (déclarée abstraite dans la super-classe). Elle définit également un attribut protégé et un constructeur, qui fait appel à celui de sa super-classe.

`public final ClassC extends ClassB {`
`  private int y, z;`
`  public class ClassC(int r, int q) {`
`  super(r);`
`  z=q;`
`  }`
`  @Override`
`  public void m1() { x--; }`
`  @Override`
`  public void m2(int a) { y=a; }`
`  public void m4(double b) { m4((int)(b+z)); }`
`}`
	`ClassC` n'est pas abstraite, mais toutes les méthodes déclarées ont déjà été définies plus haut dans la hiérarchie. Elle **redéfinit** l'implémentation de deux méthodes, cependant (notez que `m1(): void` fait appel à `x`, attribut hérité), et **surcharge** la méthode `m4` avec une méthode prenant un `double` en paramètre. Cette méthode fait appel à `m4(int): void`, qui est héritée. On ne pourra pas hériter de cette classe, ses attributs sont donc privés.

`public class ClassD extends ClassB`
`  implements IB {`
`  protected int w;`
`  public ClassD(int delta, int omega) {`
`  super(delta+omega);`
`  w = omega-delta;`
`  }`
`  @Override`
`  public int m3() {return w+super.w; }`
`  @Override`
`  public void m4(int x) { super.m4(x-w); }`
`}`
	`ClassD` hérite de `ClassB` et se conforme à `IB`, elle doit donc donner une implémentation pour `m3(): int`. Elle déclare un attribut `w` qui porte le même nom qu'un attribut dont elle hérite, elle accède à ce dernier avec le mot-clé `super`. Elle redéfinit aussi `m4(int): void`, en faisant appel à la définition de cette méthode établie dans sa super-classe avec le même mot-clé.

2 *Exercice – Étude de cas à terminer*

Sujet	Il s'agit de terminer la conception d'une **application de sondage**. De multiples utilisateurs peuvent utiliser cette application sans connexion. Tout utilisateur peut aussi créer un sondage, en entrant une adresse électronique valide et le nom de son sondage. Il peut ensuite ajouter des questions, soit à choix multiples, qui doivent comprendre chacune un texte et une liste de réponses possibles, soit à choix unique (même chose mais il n'y a qu'une réponse possible), soit à réponse courte (texte libre), soit enfin un ensemble de réponses à classer par ordre de préférence. Quand il a terminé, il peut valider ce sondage, et il reçoit un lien à partager. Les autres utilisateurs qui utilisent ce lien peuvent répondre au sondage, question par question, et valider leur entrée à la fin. Tout créateur de sondage peut à tout instant voir l'ensemble des réponses à son sondage.
Acteurs	Deux catégories d'utilisateurs, les sondeurs et les sondés : `:POLL_CREATOR` et `:POLL_TAKER` par exemple.
Événements externes	Pour les sondeurs : — Créer un nouveau sondage, — Ajouter une question, — Valider un sondage, — Demander l'affichage des résultats du sondage. Pour les sondés : — Demander à répondre à un sondage, — Répondre à une question, — Valider les réponses au sondage.
Événements résultats	Vers les sondeurs : — Le lien vers un sondage nouvellement créé, — l'affichage des résultats actuels d'un sondage, — une notification quand un sondé a terminé le sondage. Vers les sondés : — L'affichage d'une question, — l'affichage d'un message d'erreur en cas de réponse incorrecte, — l'affichage de la demande de validation de l'ensemble des réponses.
Cas d'utilisation	
Classes – brouillon	On cherche à représenter les entités métier, pas les éléments d'interface utilisateur. On peut penser à une classe pour le sondage, et une pour chaque catégorie de questions (les réponses seront des données dans certaines classes) : 

Le travail consiste à reprendre le diagramme de classes en fonction du sujet, pour commencer à en préciser les opérations et données, et à chercher à éviter au maximum la redondance en utilisant au mieux de possibles interfaces, classes abstraites et héritage.